COSC 39: Theory of Computation

Problem 1. Parsing regular expressions.
Can the task of parsing regular expressions itself be implemented using a regular expression? This property is known as universality—the language is powerful enough to recognize its own structure.
We can treat the collection of regular expressions itself as a language RegExp, containing words that are valid regular expressions over the binary alphabet $\{0, 1\}$. To write down a regular expression in RegExp, in addition to 0 and 1 we need a few extra symbols; so we define the alphabet set for RegExp to be

$$\Sigma' := \{0, 1, +, *, (,)\}.$$

(a) Prove that the language RegExp itself is not regular. In other words, we cannot construct a parser for valid regular expressions using only DFAs/NFAs.

(b) What if we disallow nested parentheses? (In other words, inside a pair of parentheses (and), no further parentheses can appear.) Prove or disprove that the language RegExp with no-nesting constraint is regular.

Problem 2. Regular or not?.
Prove or disprove that each of the languages below is regular (or not). Let Σ^+ denote the set of all nonempty strings over alphabet Σ; in other words,

$$\Sigma^+ = \Sigma \cdot \Sigma^*.$$

(a) $\{\, wxw^R : w, x \in \Sigma^+ \,\}$

(b) $\{\, ww^R x : w, x \in \Sigma^+ \,\}$

Hint: To prove that a language L is regular, construct an NFA that recognizes L; to disprove that L is regular, construct a fooling set and argue that the construction is correct.